The music system

A study guide to eyeMusic's audio: devices and buses, musical time, the Band and the Eye Band, how everything is born and torn down, and where latency comes from in the boids level.

§	Section	You will be able to answer
0	The concept at a glance	The whole system on one page: a mind-map of the sections.
1	The big picture	Which object owns the music, and which only borrow it?
2	Audio devices and connections	Which player feeds which bus, what effects sit where, and what reaches the speakers?
3	Musical time	Where 'now' comes from, how steps fire, and why tempo changes never jump.
4	The Band	What happens on every sixteenth, and what waits for a bar line.
5	The Eye Band	How an eye touching a wall becomes a part dropping out.
6	Lifecycle	When each piece is created, what keeps running across levels, what tears it down.
7	Latency in the boids level	Every hop between a press or a grid step and the ear, what is compensated and what is not.
8	File map and glossary	Where to look next.

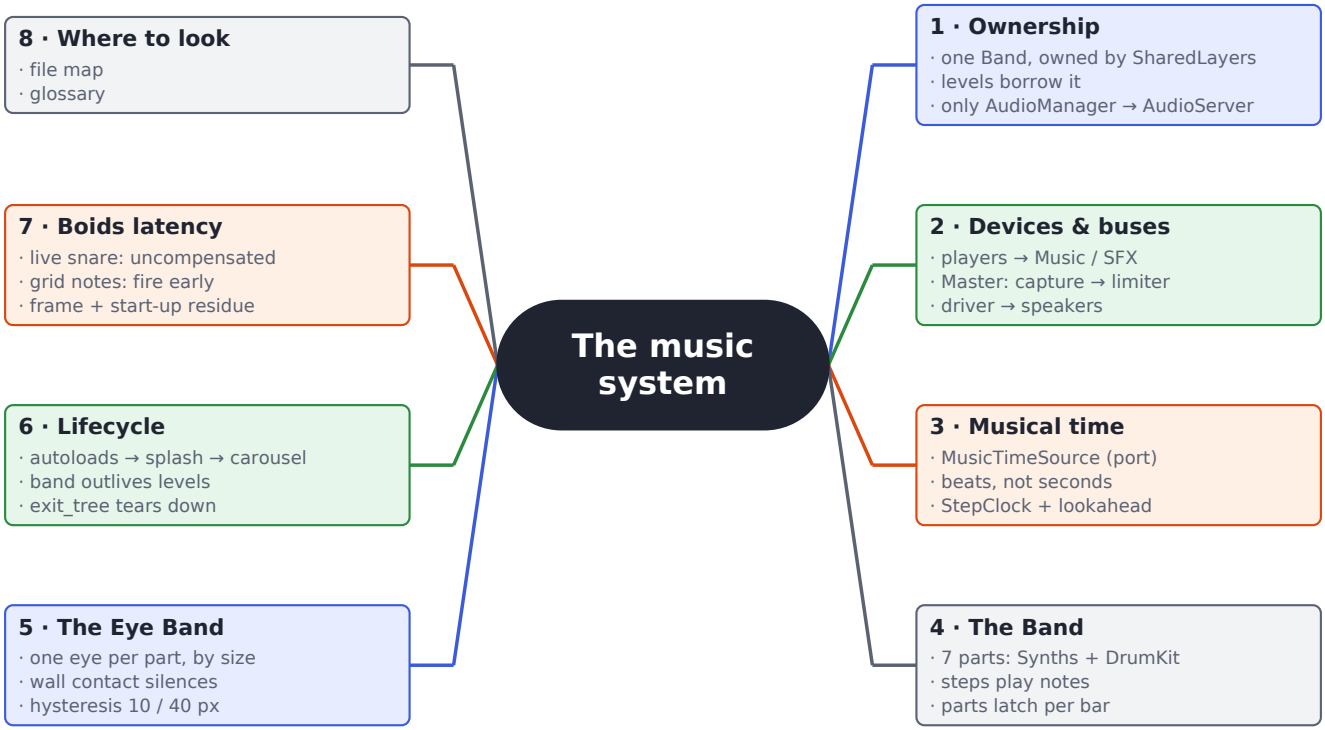
How to use it. Each section ends with a ruled box for your notes; the last pages are dot grid for sketches. File references are repo-relative paths — open them next to the guide. Numbers marked *measured* come from the repo's own READMEs; numbers marked *estimate* are typical values to verify on your device.

Written from the code on branch `main` at commit 4d229f4 (Spout: one level, with bubbles...). Engine: Godot 4.4.

✎ **What I want to understand / questions before I start**

The concept at a glance

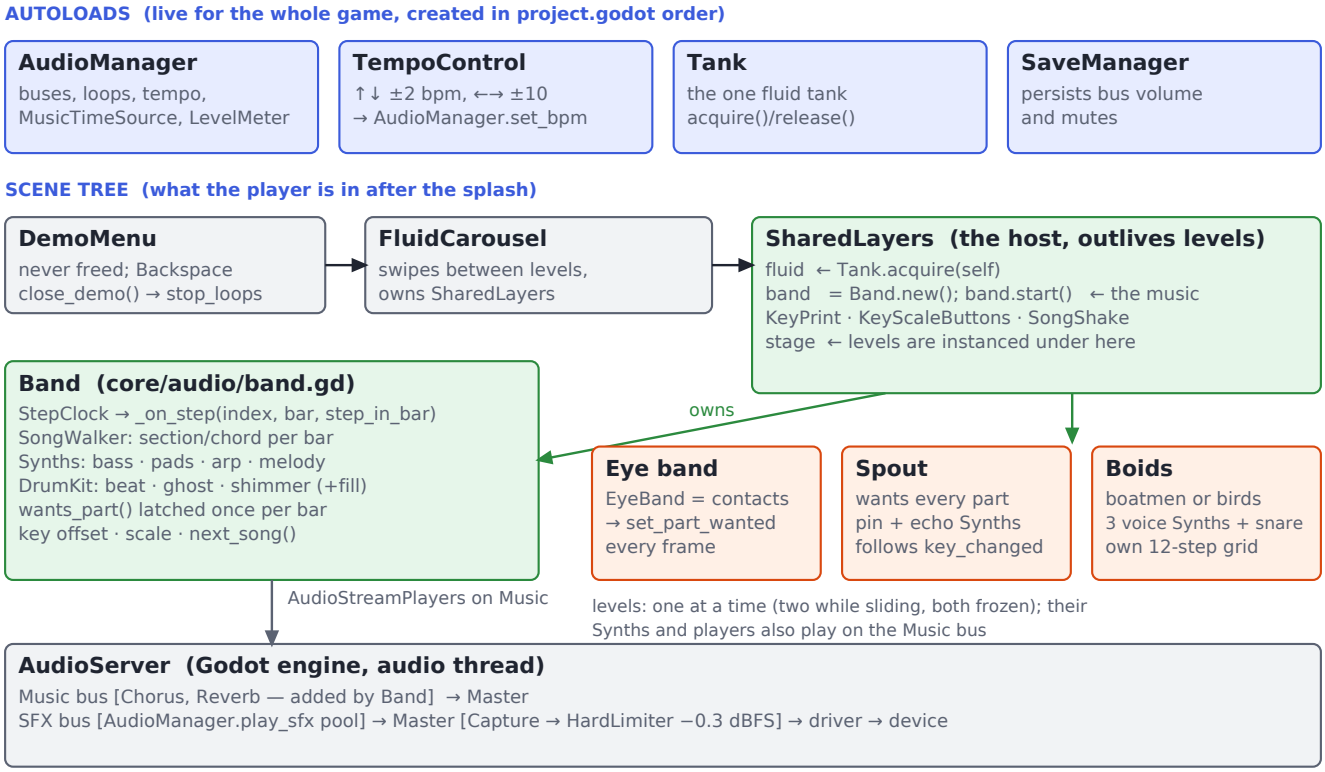
Everything in this guide hangs off one idea: a single band, kept on a single clock, that levels borrow. Read the map clockwise from the top; each branch is a section.



✎ **Notes — my own map: what connects to what?**

1 · The big picture

There is exactly **one** piece of music playing in the carousel: a `Band` owned by `SharedLayers`, the host that every level is a foreground over. Levels never start a clock, set a tempo or add bus effects; they only *borrow* the band — choose which parts they want, move its key, follow its signals, and play their own notes on top in its key.



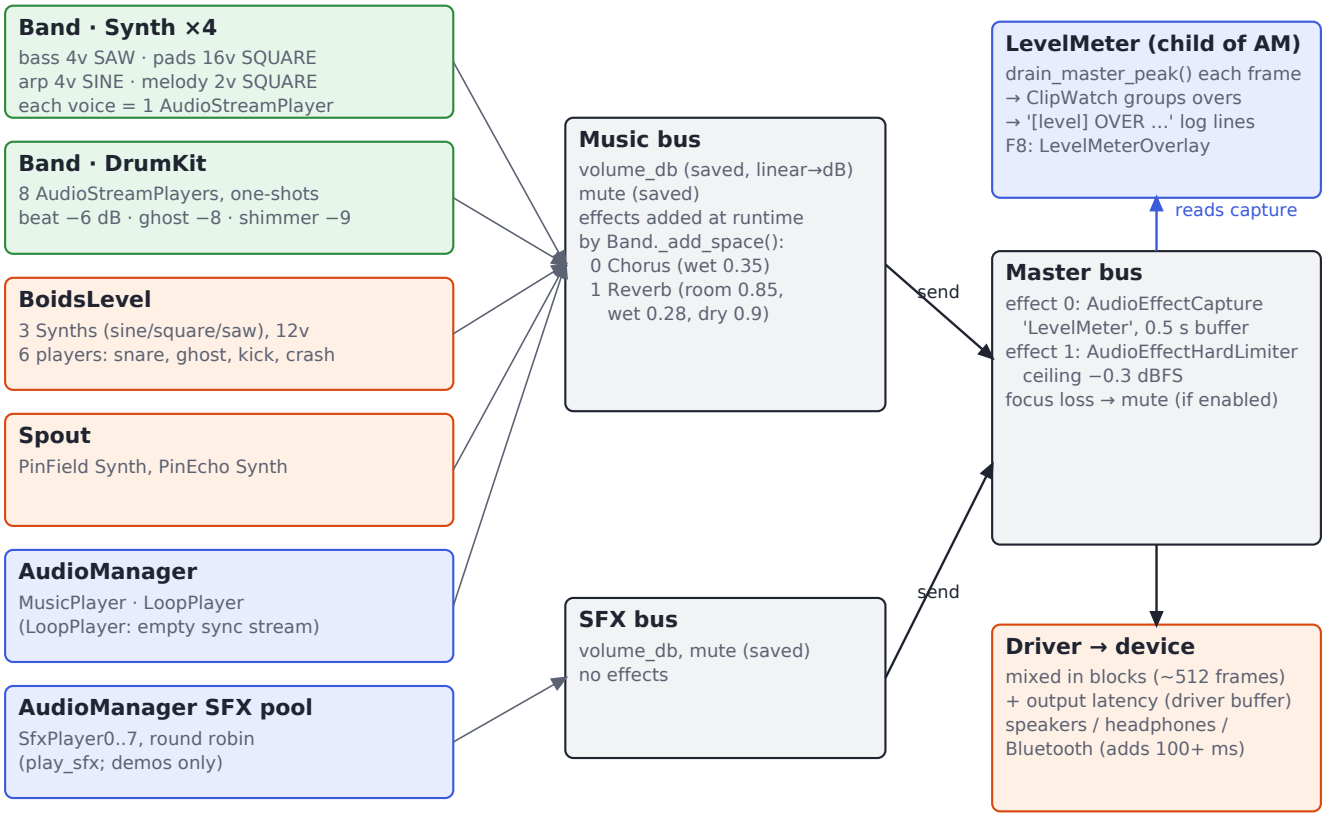
Three rules that explain most of the design:

- **Only `AudioManager` talks to `AudioServer`.** Everyone else asks it to play, set a volume, add an effect, or tell them the time.
- **Musical time is a port.** Everything rhythmic asks `AudioManager.get_music_time_source()` for beats. Swap the adapter and the whole game's timing changes in one place.
- **Musical position is beats, not seconds.** Tempo only changes how fast beats accumulate from now on, so a tempo nudge never moves the bar you are in.

✎ **Notes**

2 · Audio devices and connections

Buses are not built in code. They come from `resources/audio/default_bus_layout.tres`, loaded by the project setting `audio/buses/default_bus_layout`. Every sound in the levels is an **AudioStreamPlayer** (non-positional) on the **Music** bus — including the boids level's snare. SFX is only used by `AudioManager.play_sfx` (the audio demo).



The signal path, end to end

Stage	What it is	Where	Notes
Voice / hit	SynthVoice: one AudioStreamPlayer playing an AudioStreamSynchronized of band-limited wavetable loops; pitch via pitch_scale, envelope via volume_db, set once per frame.	core/audio/synth_voice.gd	No per-sample script. Pitch/volume change once per mix block.
	DrumKit hit: an AudioStreamPlayer playing a cached AudioStreamWAV one-shot (DrumSynth, rendered once per sample rate).	core/audio/drum_kit.gd	Pool of 8, steal oldest.
Music bus	Chorus then Reverb, added by Band._add_space() when the band starts, removed in Band._exit_tree.	core/audio/band.gd	Bus volume/mute persisted by SaveManager.
Master	Capture ('LevelMeter', 0.5 s) → HardLimiter (−0.3 dBFS).	default_bus_layout.tres	Limiter is a safety net; capture sees pre-limiter level.
Mix	AudioServer mixes all buses on the audio thread, block by block.	engine	Script play() calls land on the next block.
Driver	Platform driver (PulseAudio/PipeWire/ALSA on Linux & Deck, CoreAudio on iOS/macOS, WASAPI on Windows). Buffers before the DAC.	engine	Reported by AudioServer.get_output_latency().

Stage	What it is	Where	Notes
Device	Speakers, headphones; Bluetooth adds its own codec buffer the engine cannot see.	hardware	iOS session category = 3 (Playback) in project.godot.

The AudioManager API you will actually use

```
AudioManager.set_bus_volume_linear(AudioManager.MUSIC_BUS, 0.7)  # 0..1, stored as dB
AudioManager.set_bus_mute(AudioManager.SFX_BUS, true)
var i := AudioManager.add_bus_effect(AudioManager.MUSIC_BUS, AudioEffectReverb.new())
AudioManager.get_bus_effect(AudioManager.MUSIC_BUS, i)   # drive it, e.g. via ParameterFader
AudioManager.get_music_time_source().get_beats()         # 'what time is it', in beats
AudioManager.get_level_meter().set_note("level", name)  # context for OVER logs
```

- **Focus mute:** losing window focus mutes Master (if `mute_on_focus_loss`), and restores exactly what was there before.
- **Metering:** bus peak readings only show the latest block, so the meter drains the Master capture every frame instead. F8 toggles the overlay; `scripts/deck.sh run NAME --meter` starts with it on.
- **Headroom:** seven parts at once sit about 3 dB under the groove demo's levels (`Band.PART_DB`, `SynthPatch.level` 0.15 default) so the limiter rarely works.

✎ **Notes**

3 · Musical time

Everything on the beat — the band's sixteenths, the boids' pulses, the ghost snares, loop layers switching on a bar — asks one object what time it is.

Piece	Job	Pure?
MusicTimeSource (port)	get_beats(), set_tempo_bpm(), get_lookahead(), start()/stop().	interface
WallClockMusicTime (adapter, in use)	Integrates beats from Time.get_ticks_usec(), restarting the integration at every tempo change. Lookahead = time_to_next_mix + output_latency.	no
ScriptedMusicTime	Beats moved by hand — tests.	yes
MusicClock	beats → bar / beat-in-bar / step; seconds ↔ beats at a tempo. Replaced (not mutated) on every tempo change.	yes
StepSequencer	Which steps are due by beats + lookahead, each once, in order. After a hitch, at most 4 overdue steps (MAX_CATCH_UP) — old ones are dropped.	yes
StepClock (Node)	Calls the sequencer in _process and emits step(index, bar, step_in_bar).	no
LoopLayerScheduler	Holds loop-layer on/off requests until the next bar line.	yes
TempoControl (autoload)	Arrow keys / d-pad → AudioManager.set_bpm(), clamped 30–180.	no

How a step fires

```
# StepClock.advance(), every frame
beats     = source.get_beats()                       # wall clock, in beats
lookahead = clock.beats_from_seconds(source.get_lookahead())
for index in sequencer.update(beats, lookahead):     # steps with start <= beats+lookahead
    step.emit(index, index / per_bar, index % per_bar)
```

Why not the player's playback position?

For a looping stream it wraps at the loop point, so a bar longer than the loop would never arrive. The wall clock climbs forever. The cost: the clock is not the audio device's clock, so it cannot be sample-accurate; it is corrected (not locked) with the lookahead. A sample-accurate adapter would be a new MusicTimeSource, not a rewrite.

Tempo changes

`set_tempo_bpm` snapshots the beats reached so far and restarts integration at the new rate, so position never jumps. Note-hold lengths are wall seconds computed at note-on time, so a note already sounding keeps the length it started with.

✎ **Notes**

4 · The Band

`core/audio/band.gd` plays one of `EyeBandSongs`' four songs live. A `Song` is data: sections of chord symbols, each listing which sections may follow (weighted). `SongWalker` lays the path bar by bar, seeded.

Part	Instrument	Patch (wave · attack · release · level · voices)	Plays
beat	DrumKit	−6 dB	song's beat pattern; fill over it on a section's last bar
bass	Synth	SAW · 5 ms · 80 ms · 0.16 · 4	bass_line symbols R/O/5/3 on the chord
pads	Synth	SQUARE · 600 ms · 1.4 s · 0.045 · 16	close voicing, re-voiced on chord change
melody	Synth	SQUARE · 80 ms · 500 ms · 0.05 · 2 (glide 60 ms)	MelodyWriter, per section
arp	Synth	SINE · 5 ms · 300 ms · 0.07 · 4	arp_pattern index into chord tones
ghost	DrumKit	−8 dB	ghost-note drum pattern
shimmer	DrumKit	−9 dB	hats/shimmer pattern

What happens on each step — Band._on_step(index, bar, step_in_bar)

```
if bar changed:
    swap in next_song if one is waiting      # song changes land on a bar line
    _latch_live_parts()                      # wants_part(part) → _playing[part]
    fill_active = last bar of section and beat is playing
chord = walker.chords_at(bar) → chord_in_bar(step) → transposed(key_offset)
_play_drums(step)                            # DrumKit.play(hit, velocity, PART_DB)
pads on chord change · bass · arp · melody   # Synth.note_on + SceneTreeTimer for note_off
stepped.emit(bar, step_in_bar)               # levels follow this
```

What waits for what

Change	Takes effect	Why
A part wanted on/off	next bar line	_latch_live_parts runs once per bar, so parts enter and leave together and on the beat.
Key offset (Q/E, Z/X, corner buttons)	next step; pads at once	chords are read every step; _revoice_pads crossfades so the old key doesn't ride out a bar.
Scale (scale ▸)	immediately, for levels	the levels' palette (flock voices, spout pins), not the song's chords.
Next song (shake)	next bar line	_swap_song: new walker, patches, drum patterns, set_bpm.
Tempo (arrows)	immediately (rate)	beats integrate at the new rate; hold lengths of sounding notes stay.

Signals to follow the band: `song_started(title)`, `key_changed(offset)`, `scale_changed(name)`, `stepped(bar, step_in_bar)`.

✎ **Notes**

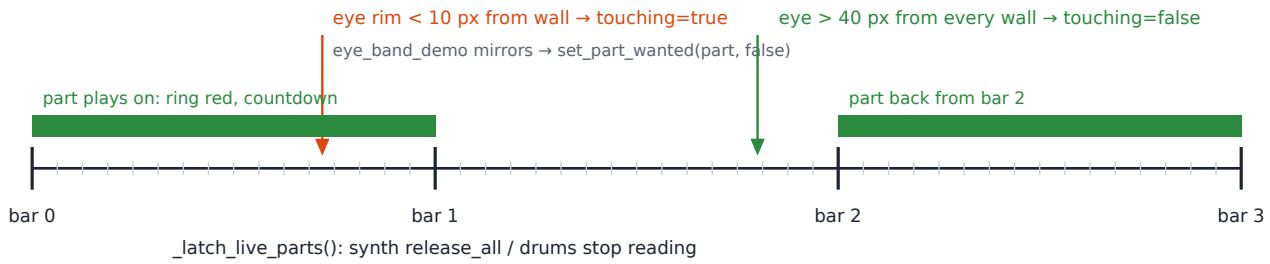
5 · The Eye Band

Seven floaty eyes drift on the tank's fluid. Each plays one part of the band — biggest eye the beat, then bass, pads, melody, arp, ghost, and the smallest the shimmer — and a part plays only while its eye keeps clear of the tank walls. The music is the reward for an arrangement you hold against a drifting medium.

Who does what

Object	Role
EyeBand (features/levels/secret_eyes/eye_band.gd)	extends Band but is **never started** in the carousel. Used only for contacts: binds eyes, gives parts by size (parts_by_size), measures wall clearance every frame with hysteresis, and answers wants_part().
eye_band_demo.gd (the level)	builds eyes, floaters, MotionInput, FluidMotionDriver; every frame mirrors _contacts.wants_part(part) into the host band's set_part_wanted(); draws rings (bright = playing, red = touching, faint = silent, countdown arc = pending).
Band (the host's)	actually makes the sound; latches the wants at each bar line.
KeyShiftInput	L1/R1 (Q/E) ±4th, L2/R2 (Z/X) ±5th → band.set_key_offset().

From touching a wall to silence



Worst case wall→silence ≈ one bar: 4 beats × 60/88 bpm ≈ 2.7 s (Glass Tide).

- **Hysteresis:** touching starts within `touch_px` = 10 px of the rim and ends only past `release_px` = 40 px, so an eye bobbing at the edge does not stutter.
- **Clearance:** `wall_clearance(pos, radius, tank)` = distance to the nearest of the four walls minus the radius; negative once the rim pokes past.
- **Centre pull:** `CENTRE_PULL` = 0.1 keeps eyes from random-walking onto walls within ~10 s — chosen by ear to keep the band playing take some effort.
- **Latency here is musical, by design:** up to a bar between the eye touching and the part dropping. It is the bar latch, not audio latency.

✎ **Notes**

6 · Lifecycle of the components

The music outlives levels. It starts when the carousel's host is built, keeps running through every swipe, and stops only when the carousel leaves the tree.

1 · Engine boot — Autoloads enter the tree: UiLook, EventBus, GameState, AudioManager, TempoControl, SaveManager, ComfortSettings, Tank. AudioManager._ready builds its players, loads saved volumes, finds the Master capture, adds LevelMeter.

↓

2 · Splash — main_scene = splash.tscn. Tank.acquire(splash) starts THE tank and prints the logo into it. No music yet. It hands over to DemoMenu with the carousel open.

↓

3 · Carousel opens — DemoMenu.open_demo(carousel) → FluidCarousel._ready → SharedLayers.new(). SharedLayers._ready: Tank.acquire(self); Band.new(); KeyPrint; band.start(); buttons; Stage.

↓

4 · Band.start() — pick song → build Synths + DrumKit → add Chorus+Reverb to Music → set_tempo(song) → configure_loop_layers([]) → play_loops() (starts WallClockMusicTime) → StepClock.

↓

5 · Level arrives — level scene instanced under stage. _ready: SharedLayers.host_of(self), take fluid + band, build its own instruments (Boids: 3 Synths + 6 players), set wanted parts.

↓

6 · Each frame — AudioManager._process (loop layer scheduler) → Band's StepClock fires due steps → Band._on_step plays notes → level _process (Boids: own sequencer) → SynthVoice envelopes.

↓

7 · Swipe — next level instanced FROZEN; both slide 1.4 s; outgoing freed (its _exit_tree releases its synths). Band, clock and tank never stop.

↓

8 · Back to menu — Carousel freed → Band._exit_tree: stop_loops (clock stops), release_all, kit.stop_all, remove bus effects. DemoMenu.close_demo also calls stop_loops. Tank sleeps.

Component	Created by / when	Lives until	Torn down by
AudioManager + players, LevelMeter	autoload, engine boot	game quits	—
MusicTimeSource (WallClock)	AudioManager field	game quits; running only between play_loops() and stop_loops()	stop_loops()
Tank (fluid)	first Tank.acquire() — the splash	game quits; sleeps when nobody holds it	release() / holder leaving tree
SharedLayers	FluidCarousel._ready	carousel freed	tree
Band, its StepClock, Synths, DrumKit	SharedLayers._ready → band.start()	carousel freed	Band._exit_tree: stop_loops, release_all, stop_all, remove effects
Music bus Chorus + Reverb	Band._add_space()	band exits	remove_bus_effect (reverse order)
A level (eyes / spout / boids)	carousel instances it under stage	next swipe completes	_arrive → queue_free; its _exit_tree releases its synths
Boids Synths + one-shot players	BoidsLevel.start() → _build_audio()	level freed	children of the level
EyeBand contacts	eye_band_demo._ready	level freed	child of the level

Component	Created by / when	Lives until	Torn down by
Note-off timers	every note_on (SceneTreeTimer)	fires once	skipped if the voice was retriggered (started_usec check)

Per-frame order matters

Godot processes nodes in tree order: autoloads first (AudioManager), then the current scene. Inside SharedLayers the Band (and its StepClock) comes before Stage, so in a frame the band's steps fire *before* the level's own _process. SynthVoice envelopes move in their own _process. Levels sliding in are PROCESS_MODE_DISABLED until they arrive, so a level's own notes are silent during a slide while the band plays on.

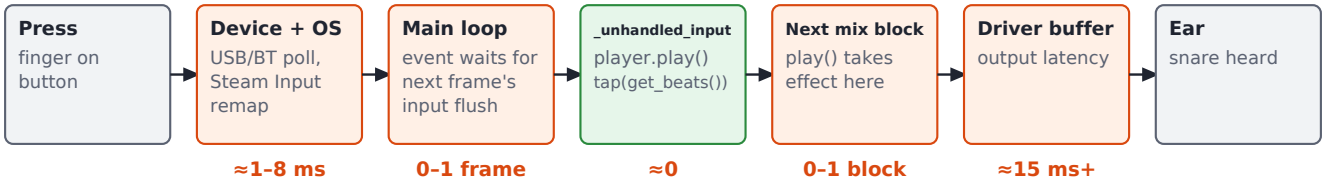
✎ **Notes**

7 · Latency in the boids level

`features/levels/boids/boids_level.gd` has four kinds of sound, and they travel two different kinds of path:

Sound	Triggered by	Path	Compensated?
Flock pulses (bird voices)	BoidsLevel._play_due → own StepSequencer on a 12-step grid	Synth.note_on → SynthVoice	Yes: fires early by mix + output latency
Ghost snare	_play_due when beats + lookahead ≥ ghost	one-shot AudioStreamPlayer	Yes, same lookahead
The band under it	Band's StepClock (16-step grid)	Synth / DrumKit	Yes, same lookahead
Live snare	_unhandled_input on press	one-shot AudioStreamPlayer	No — can't be
Scatter crash + kick	ScatterCharge fills in _process	one-shot	No (not on the grid)

A. The live snare: press → ear

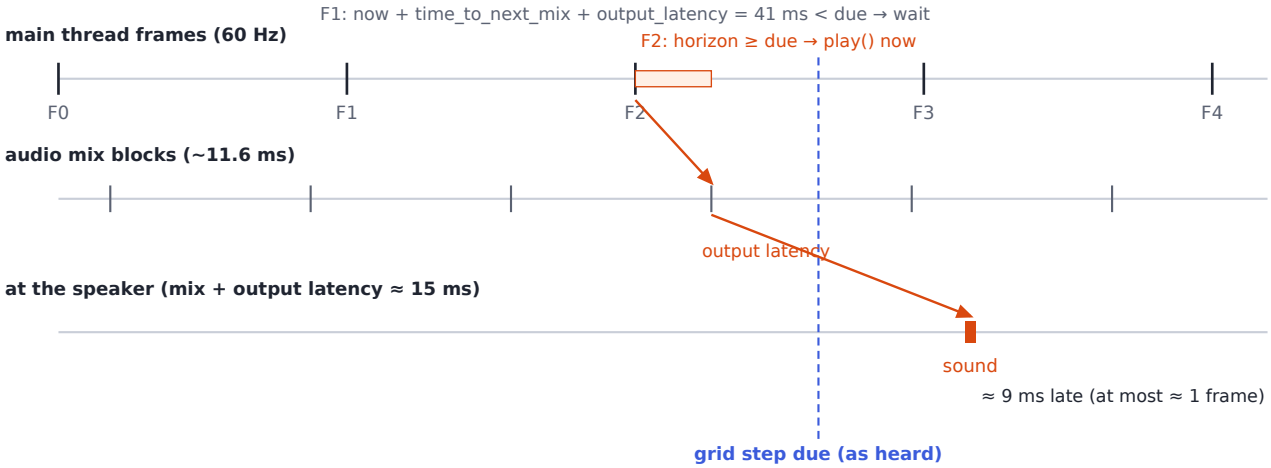


1 frame = 16.7 ms @60 Hz (11.1 @90 Hz) · 1 mix block ≈ 10.7–11.6 ms (512 frames @48/44.1 kHz)
Output latency: Godot asks the driver for 15 ms by default; the real figure is whatever the driver reports.

None of these hops is compensated for the live snare — it cannot be: nobody knows a press is coming. Wired total ≈ 20–50 ms; Bluetooth adds 100–250 ms.

The tap's position is read at the green box, so it is late by the two hops before it (device + frame wait) — that is what tap_offset_seconds is for.

B. Scheduled sounds: grid step → ear



The step fires on the first frame whose horizon has passed it, so it is heard 0 … 1 frame late (never early). Synth notes add their own start-up — see below.

✎ Notes — snare vs scheduled sounds

The latency budget, hop by hop

#	Source	Size	Hits	Compensated by	Where in code
1	Input device + OS (USB/BT poll, Steam Input desktop-layout remap on the Deck)	≈1–8 ms (estimate; BT pads more)	snare, tap reading	—	outside the game
2	Waiting for the next main-loop iteration to deliver the event (input is flushed once per frame)	0–1 frame (0–16.7 ms @60 Hz)	snare, tap reading	—	engine; _unhandled_input
3	Frame quantisation of scheduled events: a step is noticed on the first frame whose horizon has passed it	0–1 frame, always late	flocks, ghosts, band	— (lookahead cannot remove it)	StepSequencer.update / _play_due
4	play() only starts at the next mix block	0–1 block (≈10.7–11.6 ms)	everything	get_time_to_next_mix() in lookahead (scheduled only)	WallClockMusicTime.get_lookahead
5	Driver output buffer, mix → DAC	≈15 ms requested (driver decides)	everything	get_output_latency() in lookahead (scheduled only)	use_output_latency_compensation
6	SynthVoice start-up: the envelope holds at silence until a _process frame sees get_playback_position() > 0, then ramps over the attack (4 ms for birds)	≈0–1 frame + attack (hypothesis — measure)	bird voices, band synth parts on idle voices	—	SynthVoice.advance (_awaiting_start)
7	Device/hardware beyond the driver (DAC, Bluetooth codec)	≈1 ms wired; 100–250 ms BT	everything	— (engine can't see it)	outside the game
8	Human: hearing a pulse and pressing	varies	tap reading	tap_offset_seconds (0 until measured)	BoidsLevel._unhandled_input

Reading the table

- **Scheduled sounds** (flocks, ghosts, band) cancel hops 4 and 5 by firing early, leaving hop 3 (0–1 frame late) and, for synth voices, hop 6. Relative to each other they stay tight, because all three use the same clock and lookahead.
- **The live snare** pays hops 1, 2, 4, 5 and 7 in full: roughly 20–50 ms wired (estimate). It therefore lands audibly behind a perfectly-timed flock pulse that was compensated.
- **The tap's position** is read at hop 2's end via `get_beats() + tap_offset_seconds / seconds_per_beat`. Since the flocks are compensated to be heard on the grid, a tap in time with what you hear registers late by hops 1–2. To cancel that, `tap_offset_seconds` would be **negative** (≈ −(device + frame wait)).
- **Ghost snares are compensated** and use a plain one-shot player, so the ghost is the tightest sound the player hears about their own playing.

Known measurements (core/audio/README.md)

- Script-triggered sixteenths at 120 bpm: 17.7 ms spread on 512-sample block boundaries; 10.3 ms with time-to-next-mix lookahead (*measured*).
- Live synth notes vs a rendered loop, before output-latency compensation: 16–20 ms behind, ~10 ms spread (*measured*). On the Deck this was a steady lag (#78), which led to adding get_output_latency() to the lookahead — still to be re-measured by ear on the Deck.

Worth checking: hop 6 may account for part of the old 16–20 ms synth lag that output-latency compensation does not remove — the DrumKit path has no start-up wait, the SynthVoice path does. Compare a ghost snare against a bird voice on the same pulse: if the voice lands consistently later, hop 6 is real.

How to measure on a device

- Loopback: record the device's output with a mic next to a button press (a click you can hear and see in the waveform), read press → snare onset. That is the full snare chain.
- Print AudioServer.get_output_latency() and get_time_to_next_mix() on the Deck; compare with the requested 15 ms.
- Tap along to a 4-flock for a minute and log reader.quantise() errors: their mean is your tap_offset_seconds (with sign).
- Try `Engine.max_fps` / vsync off, and 90 Hz on an OLED Deck: hops 2, 3 and 6 shrink with the frame time.

✎ **Notes — my latency measurements**

8 · File map and glossary

File	What to look for
autoload/AudioManager.gd	players, bus API, loop stack, time source, set_tempo/set_bpm
autoload/TempoControl.gd	live tempo keys
resources/audio/default_bus_layout.tres	Master/Music/SFX, capture + limiter
core/audio/time/…	MusicTimeSource, WallClockMusicTime (lookahead), ScriptedMusicTime
core/audio/music_clock.gd, step_sequencer.gd, step_clock.gd	beats → steps; firing steps
core/audio/band.gd	the band: parts, _on_step, latching, key, scale, next_song
core/audio/synth*.gd	Synth pool, SynthVoice (envelope, start-up wait), wavetables
core/audio/drum_kit.gd, drum_synth.gd, step_pattern.gd	live drums and notation
core/audio/song.gd, song_walker.gd, chord.gd, melody_writer.gd	songs as data
core/audio/level_meter.gd, clip_watch.gd	overs and headroom logs
core/audio/rhythm/…	Rhythm, RhythmTable, TapReader (boids)
features/levels/carousel/shared_layers.gd, fluid_carousel.gd	host and level lifecycle
features/levels/secret_eyes/eye_band.gd, eye_band_demo.gd	the eye band
features/levels/boids/boids_level.gd	flock voices, snare, ghosts, tap reading
core/audio/README.md, …/boids/README.md, docs/boids-rhythm-scope.md	design reasoning

Term	Meaning here
beat	Unit of musical position; MusicTimeSource counts these.
step	Grid slot: 16 per bar for the band, 12 per bar (LCM of 3, 4, 6, 4) for the boids.
lookahead	Seconds a scheduled event fires early: time to next mix + output latency.
mix block	Chunk the audio thread mixes at once (~512 frames); script changes land on these.
output latency	Driver buffering between a mixed block and the DAC.
part	One of beat, bass, pads, melody, arp, ghost, shimmer.
latch	Band copying wants into what plays, once per bar.
host	SharedLayers: tank + band, outliving levels.
ghost	Quiet snare a bar after a tap, on the pulse the tap was read as.

✎ **Notes**

Sketches — signal flow

Sketches — free notes